Day 12: SIEM Data Exploration

Installing Splunk Enterprise, Indexing Authentication Data, and Building a Time-Filtered Visualization

Field	Details
Intern Name	Amrita Pathak
Date	14 September 2026
Internship Track	SOC (Security Operations Center)
Task Deadline	14 September 2026, 11:59 PM IST

Table of Contents

Table of Contents 2
Executive Summary 3
1. Objective 3
2. Data and Environment Declaration 3
3. Tools and Environment 4
4. Key Terms 4
5. Why This Task Matters 4
6. SPL Query Reference 5
7. Step-by-Step Execution 5
7.1 Installing Splunk Enterprise on Kali 5
7.2 Loading Authentication Data into Splunk 11
7.3 Recovering from an Interrupted Session 14
7.4 Searching, Time Filtering, and Visualizing 16
8. Results Summary 18
9. Findings 18
9.1 The deduplicated Splunk count matched the command-line result exactly 18
9.2 A known-good dataset turned an error into an immediately solvable one 18
9.3 The correct recovery action was the least destructive one available 19
9.4 The interruption was resolved deliberately, not just waited out 19
10. Conclusion 19

Executive Summary

This task installed Splunk Enterprise's free-licensed edition on the Kali lab VM, indexed a small authentication log, and used Splunk's Search Processing Language (SPL) to filter by time and build a saved, time-scoped visualization, the SIEM equivalent of the command-line filtering practiced in an earlier exercise. Rather than downloading Splunk's official tutorial dataset, the same synthetic SSH authentication log created and analyzed previously with grep, awk, and sort was reused here, letting the two exercises be directly compared on identical data.

The exercise did not go smoothly end to end, and that is reported here honestly rather than smoothed over. Midway through, the Kali VM froze completely under memory pressure while running a search. Recovering from the resulting hard shutdown required giving the VM more memory to work with, and, separately, caused the source file to be indexed twice, producing a duplicate event count that had to be identified and corrected in the search itself rather than by deleting already-indexed data.

Once resolved, a time-filtered search against a 30-second window returned exactly 14 failed authentication events, matching the original grep -c count from the earlier command-line exercise precisely, and a column chart visualization ranking usernames by failure count was built, saved as a Splunk Report, satisfying the task's requirement for either a visualization or a saved search with both.

1. Objective

Install and configure a working Splunk Enterprise instance, index authentication-related log data, search for authentication events, apply time-based filtering, and produce one saved visualization or saved search from the results.

2. Data and Environment Declaration

The data indexed in this task is sample_auth.log, the same synthetic SSH authentication log created by hand for an earlier command-line filtering exercise. It is not captured network traffic. Reusing it here was a deliberate choice: since its exact contents and expected pattern counts were already known and verified, any discrepancy in Splunk's results could be immediately recognized as either a genuine tool difference or a data-handling problem, which is precisely what happened when a duplicate upload was caught by comparing Splunk's count against the already-known correct total.

Splunk Enterprise was installed under its free trial license (500MB/day indexing, converting to a permanently free license after 60 days), running locally on the Kali VM rather than through a cloud-hosted trial, so that the full install and configuration process could be demonstrated as part of this task, not just the search and visualization steps.

3. Tools and Environment

Item	Detail
Endpoint used	Kali Linux VM (kali)
Platform installed	Splunk Enterprise 10.4.3, free trial license, Linux .deb package
Data source	sample_auth.log, synthetic SSH authentication log from an earlier exercise
Tools used	Splunk web interface, Splunk Search Processing Language (SPL)
Final result	Report Failed Login Attempts by Username, saved with a time-filtered search and column chart

4. Key Terms

- **SIEM (Security Information and Event Management)**: a platform that centralizes log data from many sources into one searchable system, letting an analyst query across an entire environment rather than reading individual log files one at a time. Splunk is one of the most widely used commercial SIEM platforms in real SOC environments.
- **SPL (Search Processing Language)**: Splunk's own query language, used to filter, transform, and summarize indexed data. It plays the same role inside Splunk that grep, awk, and sort played on the command line in an earlier exercise, just expressed as a single pipeline-style query instead of several chained shell tools.
- **Index**: the internal storage location where Splunk keeps ingested data once it has been processed and made searchable. Uploading a file does not simply copy it, Splunk reads its contents into the index as individual, searchable events.
- **Source type**: a label Splunk assigns to describe the format of incoming data, used to help it parse timestamps and event boundaries correctly. A custom source type can be created for data, like this exercise's log, that does not match one of Splunk's built-in formats.
- **Report (saved search)**: a search query saved under a name so it can be re-run later without retyping it, Splunk's current term for what was historically called a saved search. Saving a search as a Report was used here to satisfy the task's requirement directly.

5. Why This Task Matters

Every previous task in this internship worked directly against raw log files using command-line tools. This task deliberately shifts to the kind of platform a SOC analyst actually spends most real working hours inside, a SIEM. The specific value this adds is worth naming directly:

- **SPL and shell filtering are the same skill wearing different clothes**: grep isolates matching lines, SPL's search terms do the same. awk extracts and counts fields, SPL's rex and stats do the same. Recognizing that these are the same underlying operations, not unrelated skills to memorize separately, is what makes moving between a terminal and a SIEM fast rather than starting from zero each time.

- **Reusing known data turns a tool difference into a verifiable fact**: because the exact expected count from this log was already established using grep in an earlier exercise, any mismatch in Splunk's results was immediately meaningful rather than ambiguous, which is exactly how the duplicate upload in this exercise was caught with confidence rather than guessed at.

- **Environment setup is itself a real, gradeable skill**: installing, licensing, and configuring a SIEM platform is routine work for a SOC analyst or engineer, not a solved problem handed to them in advance. The installation and recovery work in this task is as legitimate a deliverable as the final search itself.

- **Resource planning is a genuine operational skill, not an inconvenience**: a VM freezing under memory pressure while multiple machines compete for limited host resources is a realistic constraint, not a contrived one, virtualized lab and test environments hit exactly this limit in professional settings too. Diagnosing it correctly and resolving it deliberately is itself part of operating this kind of environment competently.

6. SPL Query Reference

Before walking through each step, this section explains every SPL element used in this task on its own, so the step-by-step section that follows can stay focused on what was run and what it found.

- `source="sample_auth.log"`: restricts a search to events that came from this specific uploaded file, Splunk's equivalent of pointing grep at one particular file rather than searching everything indexed.

- `"Failed password"`: a bare quoted term in SPL searches the raw text of every event for that exact phrase, the same effect as grep's pattern matching.

- `| dedup _raw`: removes events whose raw text is an exact duplicate of one already seen, keeping only the first occurrence of each. Used here specifically to correct a real duplicate upload, covered in Section 7.3.

- `| rex field=_raw "for (?<user>\S+) from (?<src_ip>[\d\.]+)"`: rex extracts new fields from unstructured text using a pattern, similar in purpose to how awk's field-splitting pulled out a username or IP by position. Here it instead matches the literal words for and from in each line and captures whatever sits between and after them into new fields named user and src_ip.

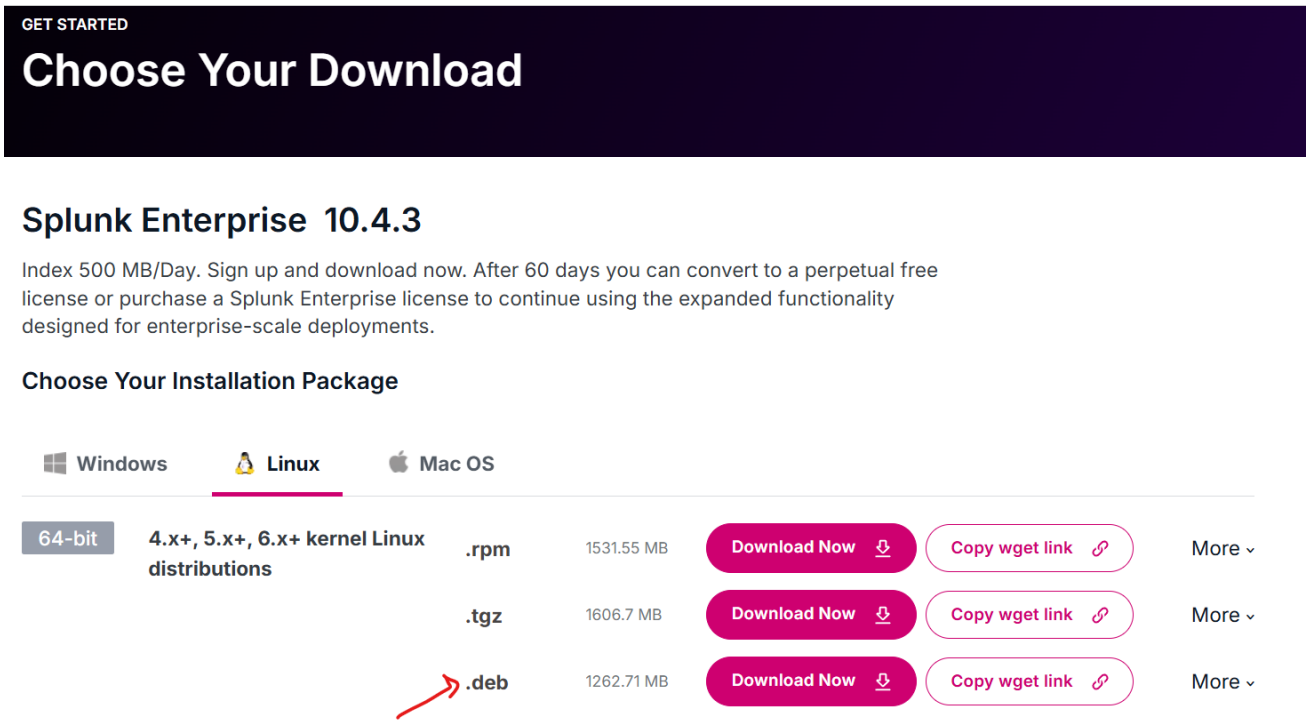
- `| stats count by user`: tallies how many events share each distinct value of the user field, SPL's equivalent of awk's associative array counting, but expressed as one command rather than a manual loop.

- Time range picker (e.g. `Custom Time`): a control in Splunk's search interface that restricts which events a search considers based on their timestamp, without needing to write the time boundaries into the query text itself.

7. Step-by-Step Execution

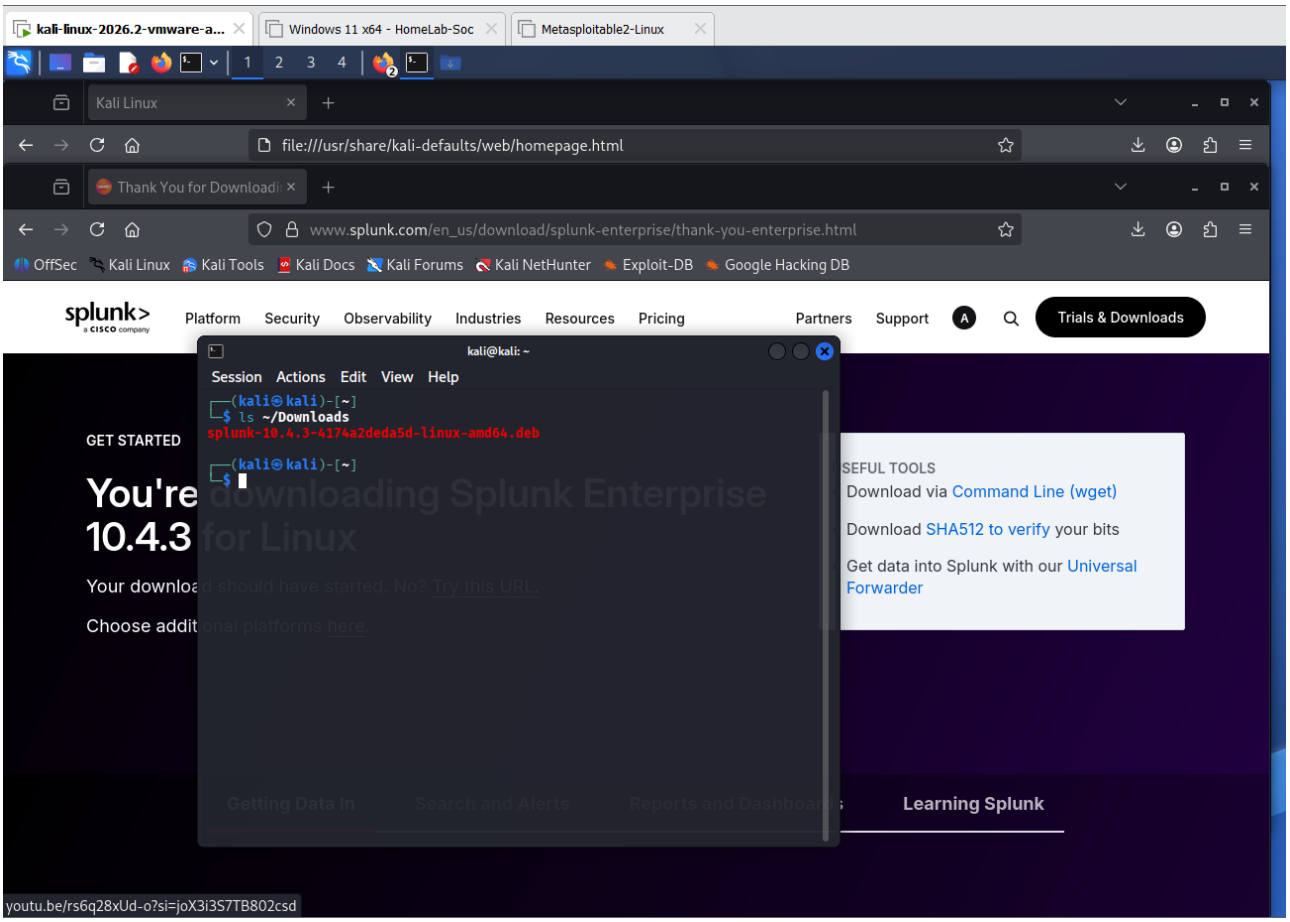
7.1 Installing Splunk Enterprise on Kali

- **Step 1 - chose the correct installation package on Splunk's download page:** after signing in to splunk.com, the download page offered several platforms and package formats. Linux was selected as the operating system, and among the Linux options, `.deb` was chosen specifically, since Kali is a Debian-based distribution and .deb is the native package format Debian and its derivatives use. The alternatives shown, .rpm (for Red Hat-based systems like Fedora or CentOS) and .tgz (a generic archive requiring manual setup), would not have installed as cleanly on Kali. The 64-bit option was selected to match Kali's architecture.



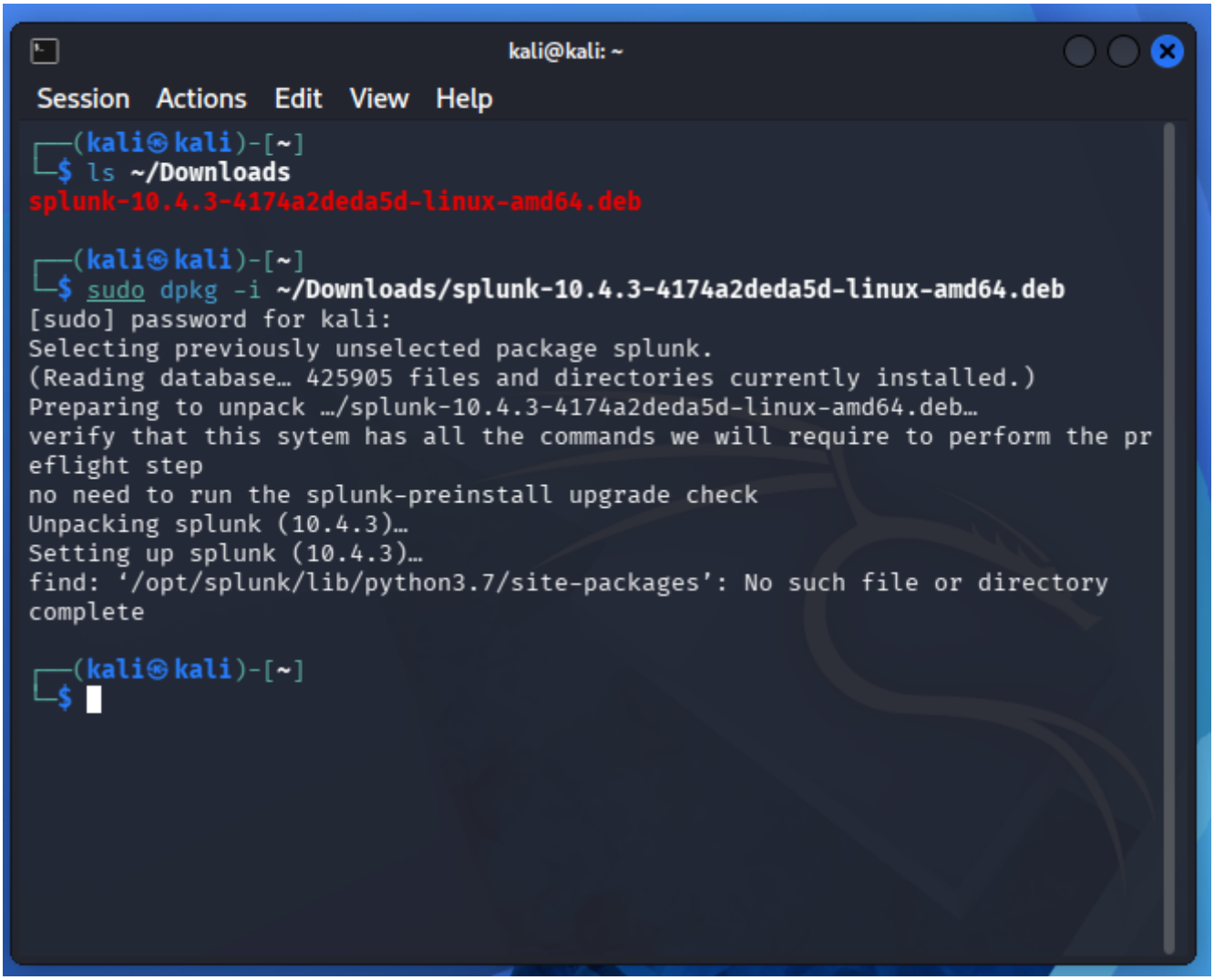
Splunk's download page, with the 64-bit .deb package selected as the correct format for a Debian-based system like Kali.

- **Step 2 - confirmed the file downloaded inside the Kali VM itself:** the download was carried out entirely inside Kali's own Firefox browser, not on the physical host computer. This mattered because moving a file of this size from a host machine into a VM afterward would have required a working copy-paste or shared-folder connection between the two, which had caused real problems in an earlier exercise. Downloading directly inside the VM avoided that transfer step entirely. Running `ls ~/Downloads` in the terminal confirmed the file, splunk-10.4.3-4174a2deda5d-linux-amd64.deb, was present in Kali's own filesystem before continuing.



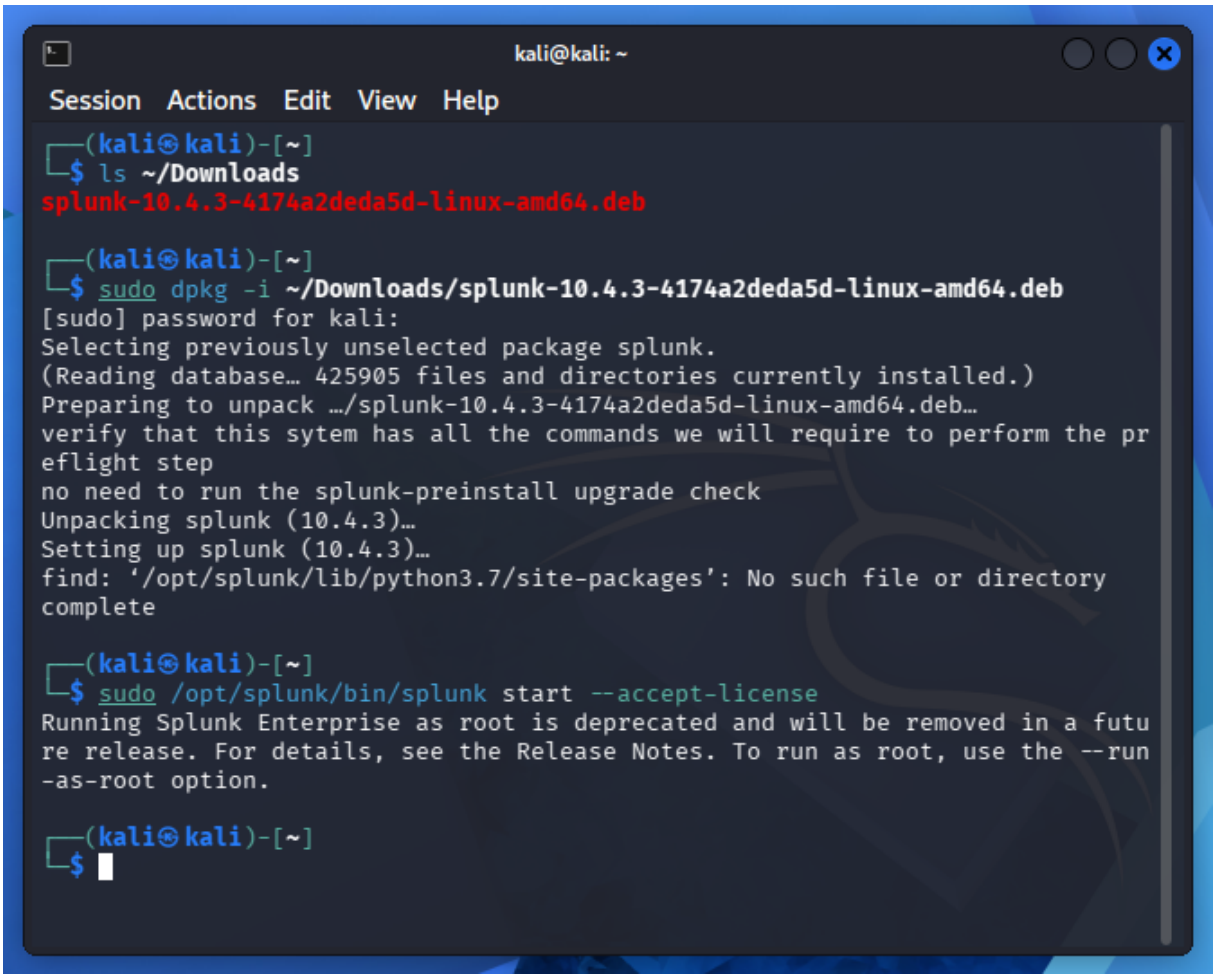
The Splunk .deb package confirmed downloaded inside Kali's own filesystem, not transferred from the host.

- **Step 3 - installed the package with dpkg:** ran `sudo dpkg -i ~/Downloads/splunk-10.4.3-4174a2deda5d-linux-amd64.deb`. dpkg is Debian's own package manager, the correct tool for installing a .deb file, and `-i` tells it to install the specified package. sudo was required since installing software system-wide needs elevated privileges. This step unpacked Splunk's files into /opt/splunk, Splunk's standard installation directory on Linux, and completed with a plain 'complete' message confirming success.



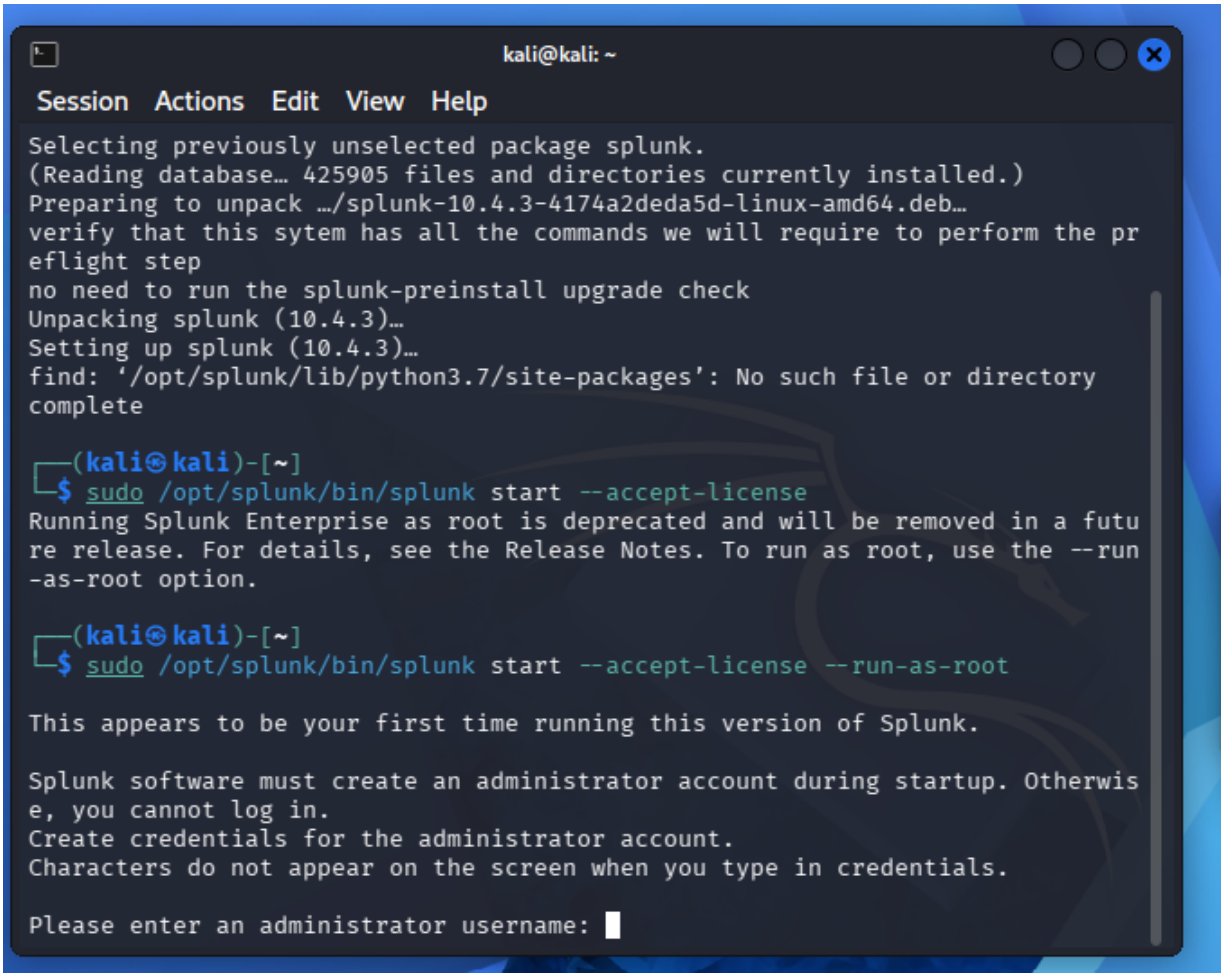
dpkg completing the Splunk installation into /opt/splunk.

- **Step 4 - first start attempt was rejected:** ran `sudo /opt/splunk/bin/splunk start --accept-license`. This is the command that launches the Splunk server process for the first time and, since it was the very first launch, would normally also trigger the license agreement and admin account setup. Instead, Splunk refused to start, printing a warning that running as root without an explicit flag has been deprecated. This happened because the command was run with sudo, which executes as the root user, and newer versions of Splunk no longer allow this silently, for safety reasons, an administrator has to explicitly acknowledge running this way.



Splunk declining to start as root without the --run-as-root flag, and explaining exactly what to add.

- **Step 5 - restarted with the correct flag and created the admin account:** ran `sudo /opt/splunk/bin/splunk start --accept-license --run-as-root`. The added flag explicitly permits running as root, which Splunk itself offers as a supported option for exactly this kind of single-user lab setup, even though a dedicated non-root service account would be the stricter choice for a production deployment. This time Splunk proceeded, asked to accept its license, then prompted for a new administrator username and password to be created on the spot, entered directly into the terminal. Splunk then finished starting on its own, printing confirmation that its web interface was ready at http://kali:8000.



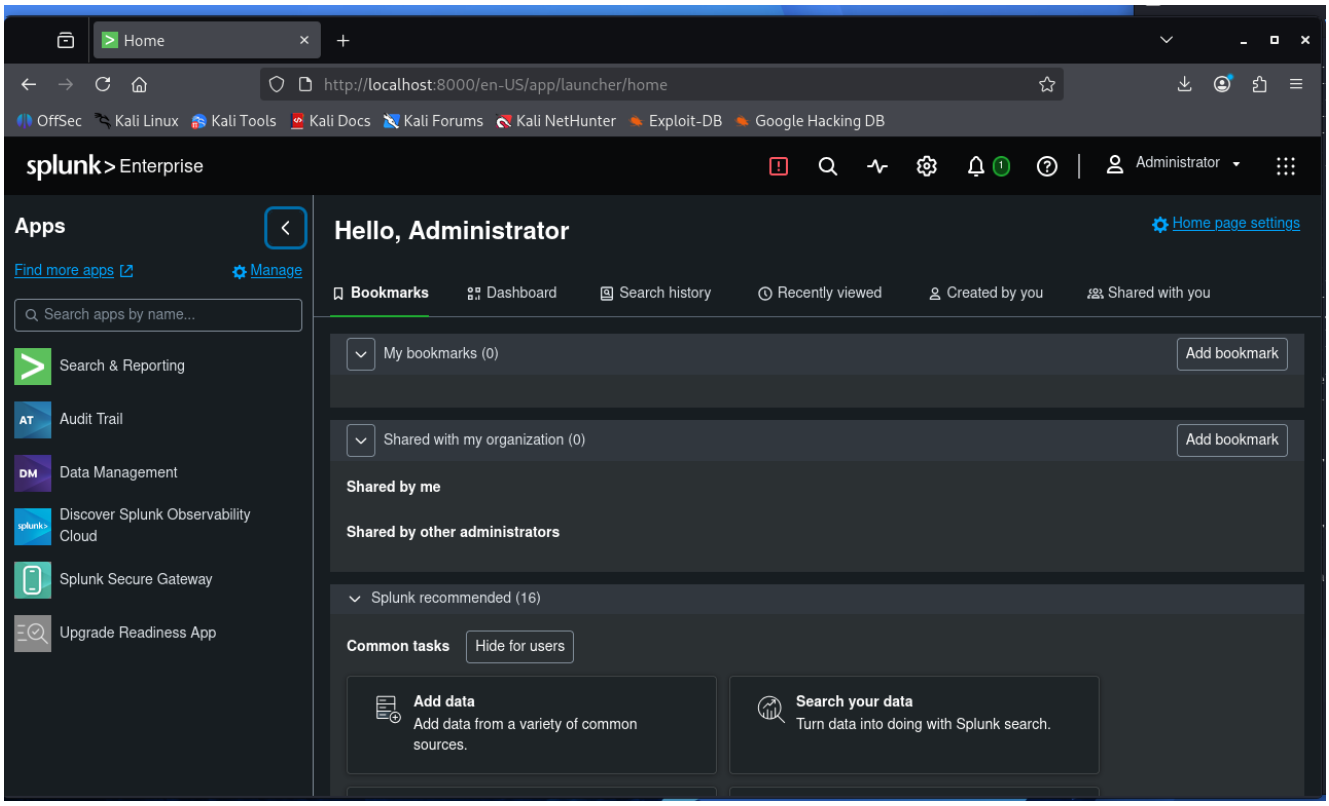
Splunk accepting the license and prompting for a new administrator account on first run.



Splunk fully started, confirming the web interface is available at http://kali:8000.

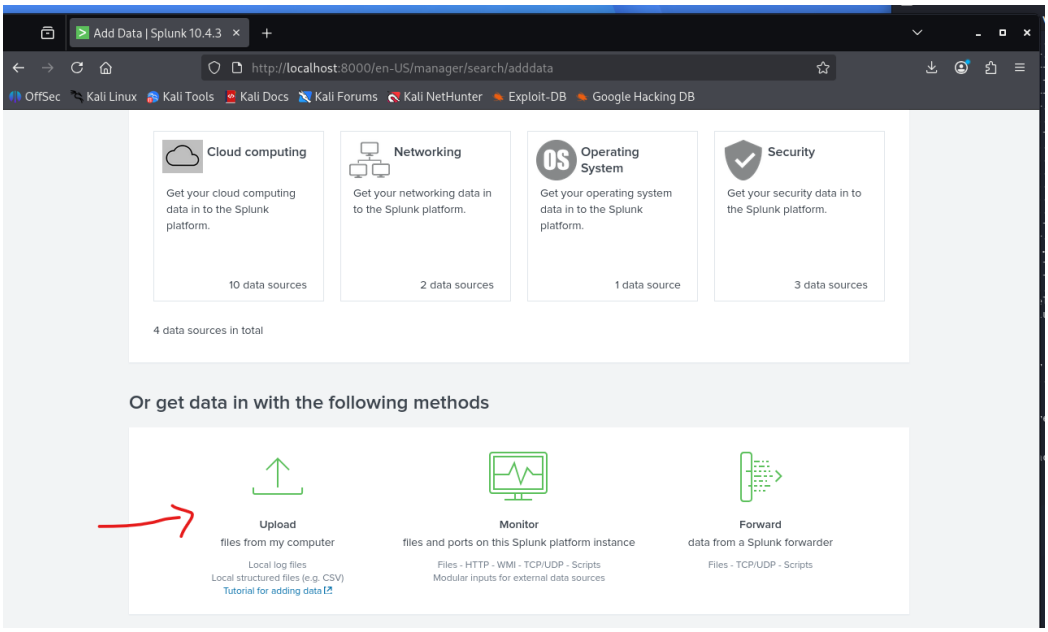
7.2 Loading Authentication Data into Splunk

- **Step 6 - logged into the Splunk web interface:** opened Firefox to `http://127.0.0.1:8000`. The plain address `localhost:8000` did not initially resolve in the browser, while the equivalent numeric loopback address, 127.0.0.1, worked immediately, both refer to the machine's own network interface, so this was a naming resolution quirk rather than a real problem. Signed in with the administrator account created in the previous step, landing on Splunk's standard home page.

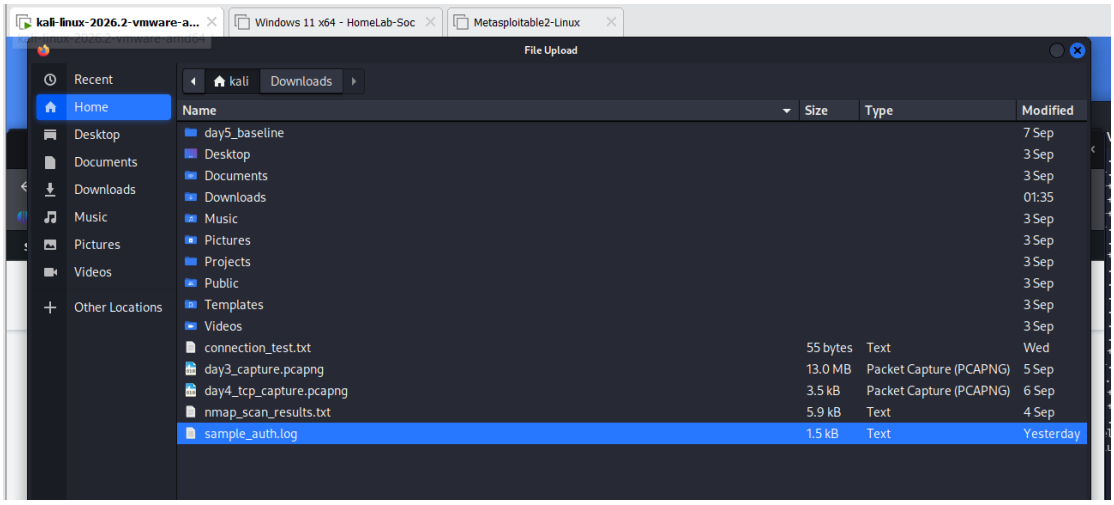


Successfully logged into Splunk Enterprise as Administrator.

- **Step 7 - selected Upload as the data input method:** from the Add Data screen, Splunk offers three ways to bring data in: Upload, for a single existing file; Monitor, for watching a live file or port continuously; and Forward, for receiving data sent from another Splunk installation. Upload was the correct choice here since sample_auth.log already existed as a complete, finished file rather than something actively growing or arriving from elsewhere.

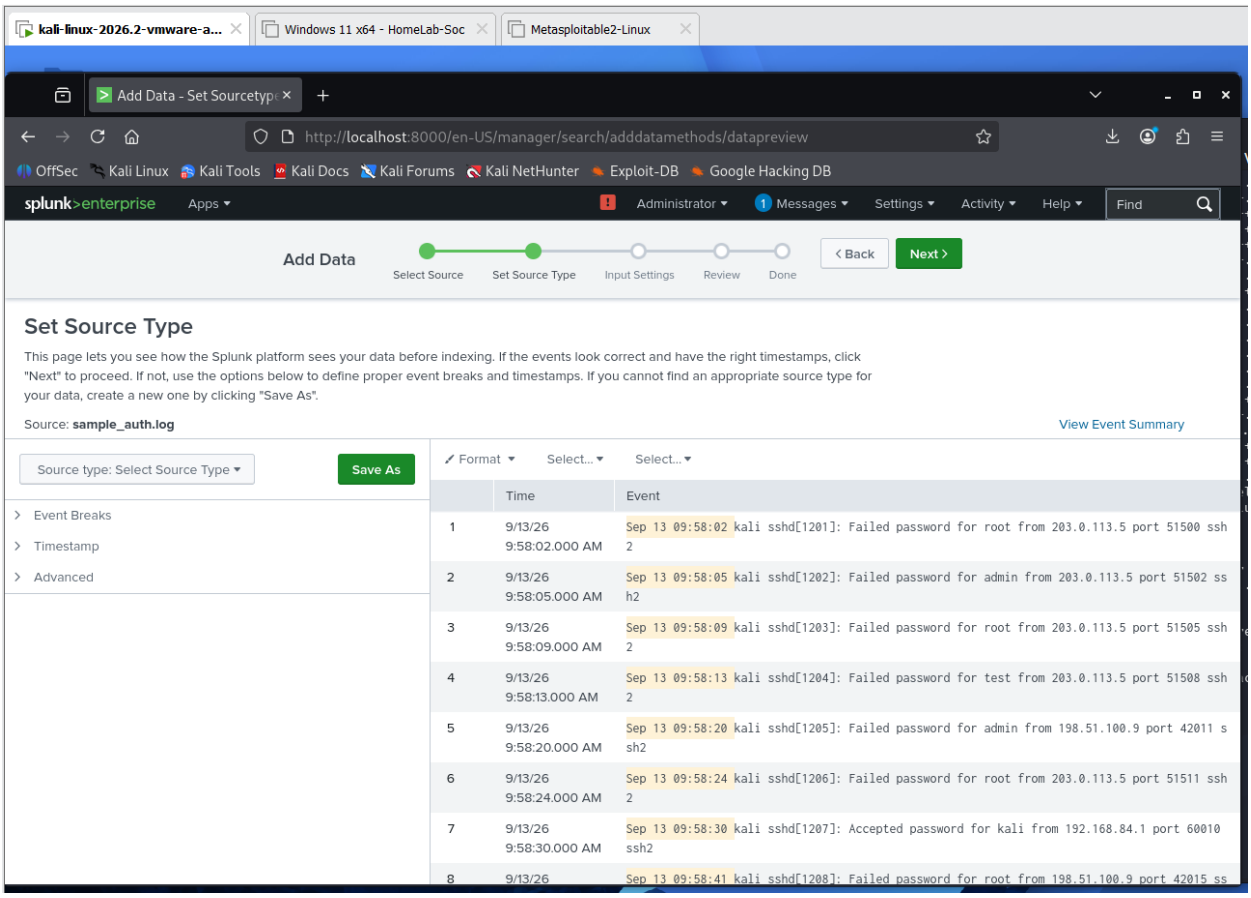


The three Add Data input methods, with Upload selected as the correct one for a single existing file.

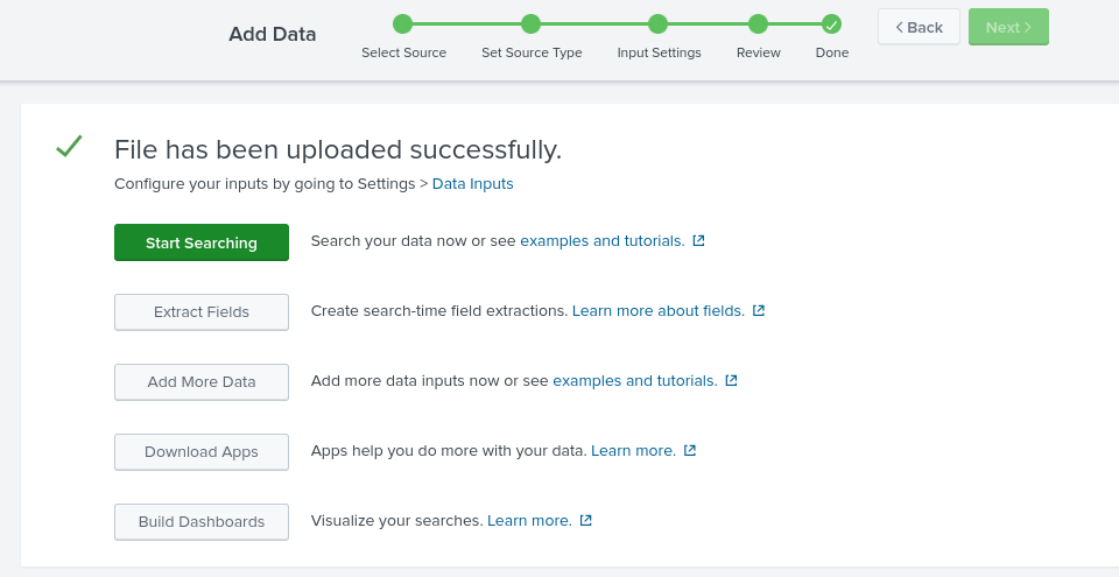


sample_auth.log located and selected from Kali's home folder for upload.

- **Step 8 - confirmed Splunk parsed the data correctly before proceeding:** on the Set Source Type screen, Splunk shows a live preview of how it intends to break the uploaded file into individual, searchable events. This preview was checked line by line rather than clicking Next automatically, confirming each log line was recognized as its own separate event and that the timestamp extracted for each one, down to the second, matched what the file actually contained. Since the file did not match one of Splunk's built-in log formats, a small custom source type named ssh_auth_sample was created and saved to describe it, this is simply a label Splunk attaches to remember how to interpret this kind of data in the future.



Splunk's preview confirming each log line parsed as a distinct event with the correct timestamp, before committing to the upload.

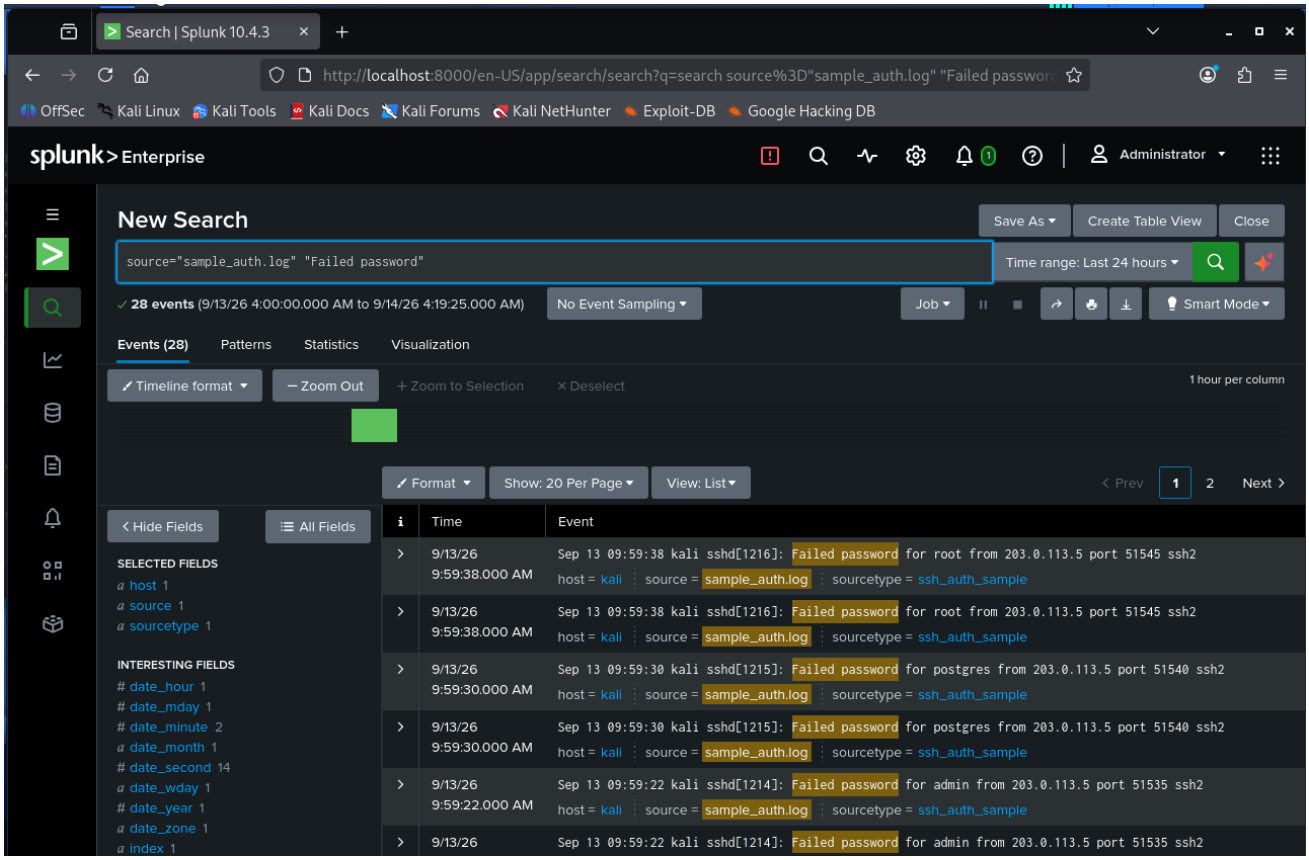


Upload completed and confirmed indexed, meaning every line is now stored and searchable inside Splunk.

7.3 Recovering from an Interrupted Session

This subsection is reported honestly rather than left out, since working through it was a genuine part of completing this task. In brief: partway through, the Kali VM stopped responding while a search was running, most likely due to memory pressure from running more than one virtual machine at once on a host with limited RAM. After confirming it was the VM itself that was frozen rather than the physical computer, Kali was restarted and given a higher memory allocation before continuing, which resolved the freeze for the rest of the exercise.

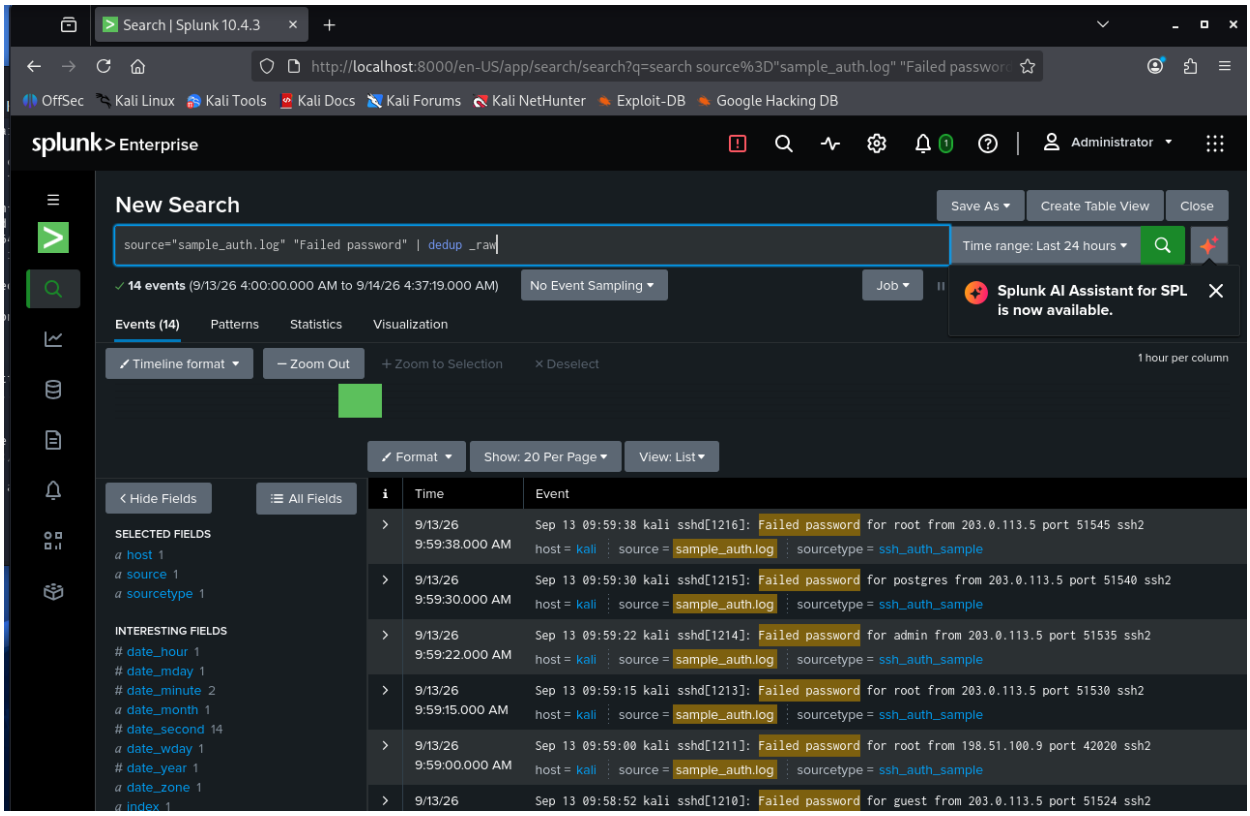
- **Step 9 - discovered the interruption had caused a duplicate upload:** after restarting Splunk and re-running the original search, it returned 28 events instead of the expected 14, with visibly duplicated entries sharing identical timestamps.



28 events returned instead of the expected 14, with duplicate entries visible at identical timestamps.

This total was immediately recognizable as wrong specifically because the correct count, 14, was already known from the original grep -c result produced in an earlier exercise against this exact file. The file had evidently been indexed twice, once before the interruption and once again after Splunk was restarted.

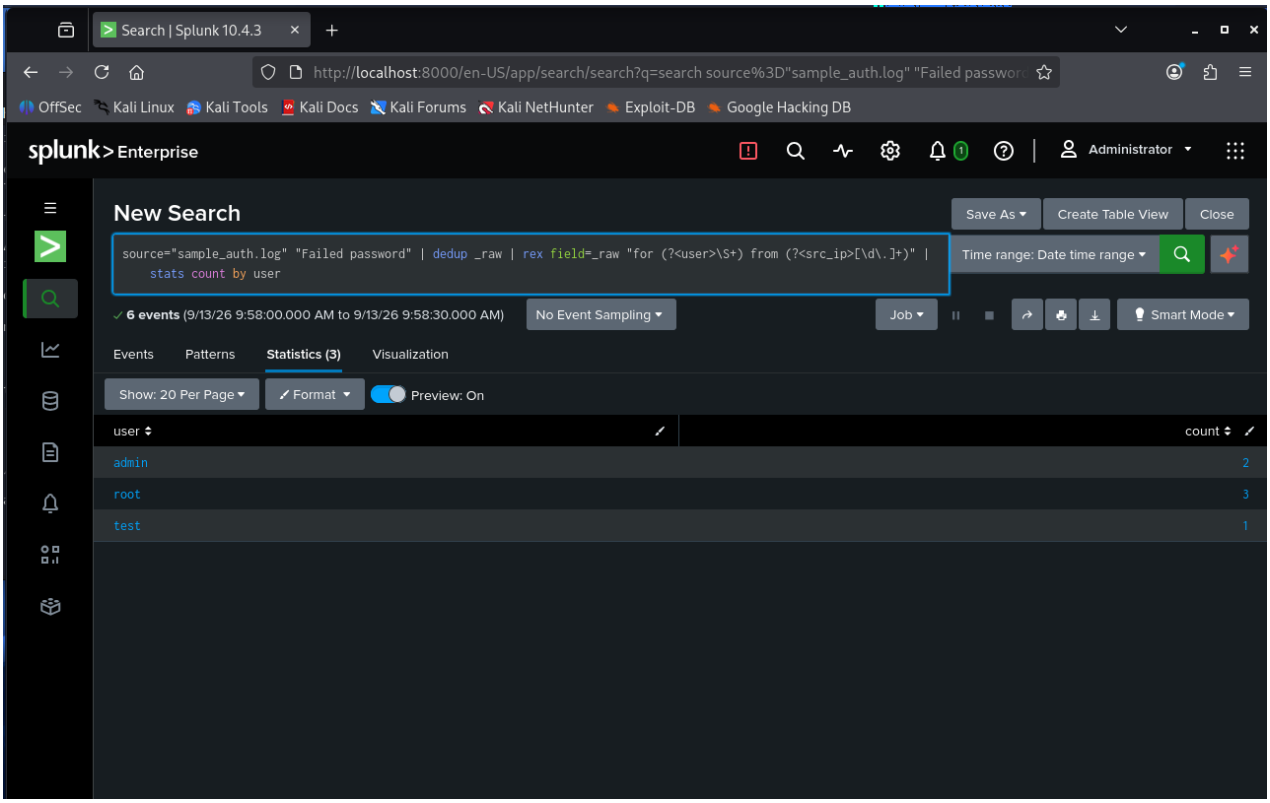
- **Step 10 - corrected the duplicate with dedup rather than deleting indexed data:** added `| dedup _raw` to the search. Deleting the duplicate copy directly from Splunk's index was considered but set aside, since Splunk requires a separate, elevated permission to be explicitly enabled before any delete operation is allowed, an unnecessary complication and risk for what dedup already solves cleanly and immediately, without altering the underlying stored data at all.



The corrected search returning exactly 14 events after adding dedup _raw, matching the original grep -c count precisely.

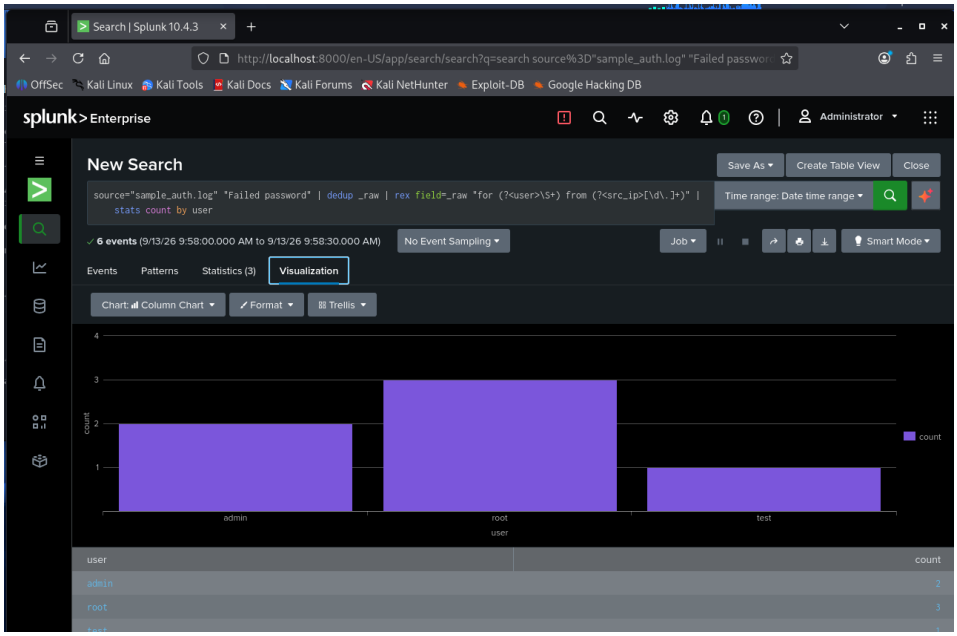
7.4 Searching, Time Filtering, and Visualizing

- **Step 11 - applied a custom time filter:** used the time range picker in the top right of the search bar, switching it from the default Last 24 hours to a Custom Time range, narrowed specifically to `9:58:00 AM` through `9:58:30 AM` on 9/13/26. This deliberately narrow window demonstrates genuine time-based filtering, only a subset of the full log's events fall inside it, rather than a default range that happens to include everything regardless.
- **Step 12 - extracted and counted usernames with rex and stats:** ran `source="sample_auth.log" "Failed password" | dedup _raw | rex field=_raw "for (?<user>\S+) from (?<src_ip>[\d\.]+)" | stats count by user`. Reading this left to right: restrict to this file, keep only failure lines, remove the duplicate copies, pull out the username and IP from each line's text, then count how many times each username appears. Within the selected 30-second window, the result was admin 2, root 3, test 1.

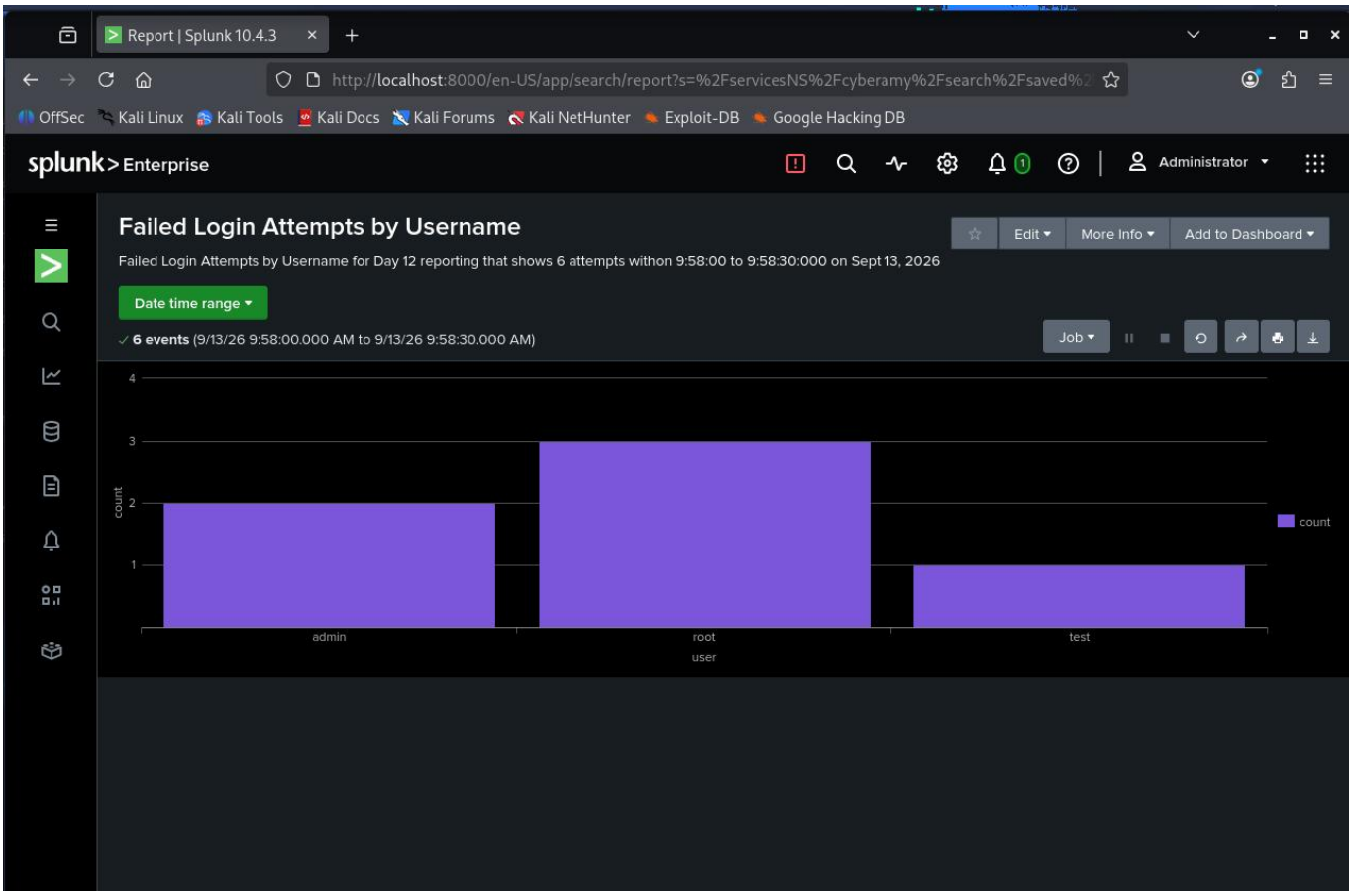


The time-filtered, deduplicated search results, tallied by username using stats count by user.

- **Step 13 - built and saved the visualization:** clicked the Visualization tab next to the search results, which automatically rendered the same statistics as a column chart, no extra configuration needed for a simple count-by-category result like this one. The search was then saved using Save As, then Report, giving it the name `Failed Login Attempts by Username`. Saving as a Report is Splunk's current equivalent of what was historically called a saved search, and doing so preserves both the query and its chart so either can be reopened later without rebuilding anything.



The results rendered as a column chart: admin, root, and test ranked by failure count within the filtered window.



The completed Report, saved and accessible independently of the original search session.

8. Results Summary

Item	Value
Total failed events (undeduplicated)	28 (duplicate upload)
Total failed events (corrected)	14, matching the original grep -c result exactly
Time filter applied	Custom range, 9:58:00 AM to 9:58:30 AM, 9/13/26
Top username in filtered window	root (3 failures)
Visualization type	Column chart, failures by username
Saved artifact	Report: Failed Login Attempts by Username

9. Findings

9.1 The deduplicated Splunk count matched the command-line result exactly

Once the duplicate upload was corrected, Splunk's search returned precisely 14 events, the same number grep -c produced against the identical file in an earlier exercise. This is a genuinely useful confirmation: two entirely different tools, a shell pipeline and a SIEM query, arrived at the identical answer against the same data, which is exactly the kind of cross-tool agreement that gives real confidence in a result.

9.2 A known-good dataset turned an error into an immediately solvable one

Reusing a log file whose correct pattern counts were already established meant the duplicate upload was caught the moment the numbers stopped matching, not discovered through careful auditing after the fact. Working against unfamiliar data would have made this same error far harder to notice at all.

9.3 The correct recovery action was the least destructive one available

Rather than pursuing Splunk's delete capability, which requires enabling an elevated permission not granted by default, dedup _raw solved the immediate problem, an inflated count, without touching the underlying index or taking on unnecessary risk. Choosing the smaller, reversible fix over a more powerful but riskier one is a reasonable operational instinct, not just a time-saving shortcut.

9.4 The interruption was resolved deliberately, not just waited out

The freeze did not resolve on its own and required a genuine diagnostic step, checking whether it was isolated to the VM or affected the whole host, before deciding on a fix. Giving the VM more memory before continuing addressed the actual cause rather than simply retrying the same setup and hoping for a different result.

10. Conclusion

A working Splunk Enterprise instance was installed, licensed, and configured on the Kali lab VM, and a known authentication dataset was indexed, searched, time-filtered, and visualized entirely within it, arriving at a result that matched an earlier command-line analysis of the same data exactly.

The more instructive part of this task turned out to be the recovery work, not the final chart. A VM freeze under memory pressure, a resulting duplicate data upload, and the decision to resolve it with dedup rather than a riskier delete operation were all real obstacles worked through directly, using the same evidence-first approach applied throughout this internship: verify what actually happened, compare it against a known-correct baseline, and choose the fix that solves the problem without introducing new risk.